

COMP3320/6464 Week 2, Lecture 1

Rhys Hawkins

Semester 2, 2024

Labs start this week

- If you haven't already done so, register for your NCI account ASAP using instructions on Wattle.
- If you have tried and run into problems, please contact comp3320@anu.edu.au

Last Lecture - Timing

- Before measuring performance and tuning we need to know what time intervals to measure and the limitations of those measurements
- Wall time and process time have different reliability issues and appropriate time intervals
- Repeated calls to a timer function can give insight into time scales

Some typical values:

- **Typical time slice interval**
 $t_s = 10\text{ms}$
- **Machine clock cycle time** = 1ns
for a 1Ghz processor
- Timer code is covered in the 1st Lab

This week

- Floating Point/Precision
- Simple Optimisations
- Assignment Background

Rationale

It is extremely important that all scientific programmers have a basic understanding of numerical data representation and error propagation ...catastrophic failure of the Ariane 5 rocket in 1996 and the inability of Patriot missile systems to reach their targets during the 1991 Gulf war were both attributed to numerical computing errors

- Ariane 5 Explosion (1996)
 - data conversion of too large number (trying to stuff a 64-bit number into a 16-bit field)
 - problem came to light as Ariane 5 was faster than Ariane 4.
- Patriot-Scud fails again (1991)
 - due to inaccurate calculation of the time since computer boot time
- Mars Orbiter loss (1999)
 - mixture of miles and meters
- See <http://www.zenger.informatik.tu-muenchen.de/persons/huckle/bugse.html>

Floating Point Numbers

- Most scientific problems are concerned with continuous phenomena, e.g time distance, velocity, temperature
 - i.e. real numbers not integers
- Computers operate in an exact discrete world
- An understanding of how floating point arithmetic is handled on computers is fundamental to scientific computing
 - all computers (CPUs) of interest have separate floating point and integer arithmetic units
- Numerical analysis deals with the design and analysis of the behaviour of algorithms for solving scientific problems
- “What every ANU CS student should know about floating point numbers” on Wattle

Sources of Error

- Before computation begins
 - modeling
 - empirical measurements - previous computations
 - mistakes
- During computation
 - approximation error (truncation or discretization error)
 - rounding error
- Accuracy of final result inevitably reflects combination of approximations, and perturbations may be amplified by nature of problem or algorithm

Approximation Error

Difference between true result (for actual input) and result that would be produced by given algorithm using exact arithmetic, e.g.

- approximation of a series

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \dots$$

$$e^x \approx 1 + x + \frac{x^2}{2}$$

- approximation of a derivative

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \text{ } h \text{ is small}$$

Rounding Error

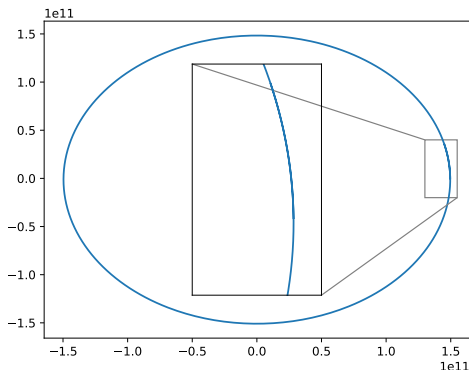
- Difference between result produced by given algorithm using exact arithmetic and result produced by same algorithm using rounded arithmetic
 - due to inexactness in representation of real numbers and arithmetic operations upon them
 - effects like $1/3 + 1/3 + 1/3 = 0.9999$

Total Computation Error

- Total Computation Error = Approximation Error + Rounding Error
 - but usually one of these is dominant

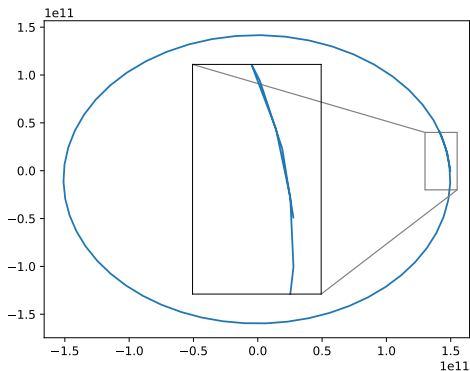
An example: Simulating Earth's Orbit about the Sun

- Compute forces due to gravity between Sun and Earth and simulate for a year
- Use a 1st order finite difference (Euler) integration
- Use 1 day as the time step



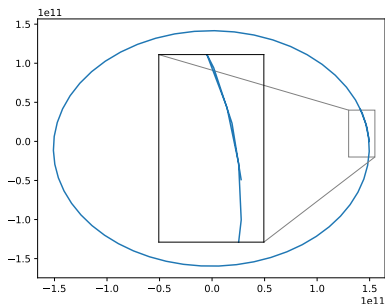
An example: Simulating Earth's Orbit about the Sun

- Use 1 week as the time step

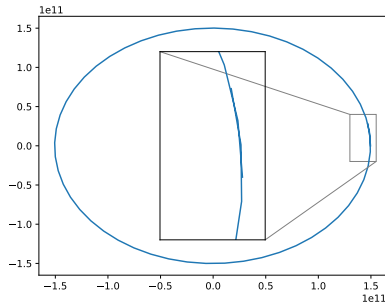


An example: Simulating Earth's Orbit about the Sun

- Use 1 week as the time step, but with Velocity Verlet instead
- Approximation error is due to method and timestep



Euler



Velocity Verlet

An example: Simulating Earth's Orbit about the Sun

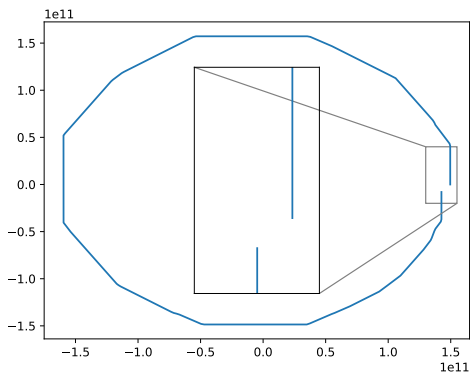
- In Euler integration, the x component is updated at each timestep using

$$x_{i+1} = x_i + v_i \Delta t$$

- What happens when the relative magnitude of $v_i \Delta t$ versus x_i tends less than 10^{-6} ?

An example: Simulating Earth's Orbit about the Sun

- Use 1 second as the time step
- Now rounding error is dominant



Absolute and Relative Error

- Absolute Error

$$\begin{aligned}\text{Absolute Error} &= \text{Approximate Value} - \text{True Value} \\ \text{Approximate Value} &= (\text{True Value})(1 + \text{Relative Error}) \\ V_a &= V_t(1 + \epsilon)\end{aligned}$$

- Relative Error

$$\begin{aligned}\text{Relative Error} &= (\text{Absolute Error})/(\text{True Value}) \\ &\approx (\text{Absolute Error})/(\text{Approximate Value})\end{aligned}$$

- True value is usually unknown
 - so we must estimate or bound error rather than compute it exactly
- Likewise relative error is often taken to be relative to approximate value rather than the true value

True Values and Precision versus Accuracy

**Accurate
Precise**



**Not Accurate
Precise**



**Accurate
Not Precise**



**Not Accurate
Not Precise**



<https://www.dnasoftware.com/our-products/copycount-qpcr-analysis-copynumber/precision-and-accuracy/>

Floating Point Numbers

- Characterised by
 - β - base or radix (generally 2)
 - t - the precision (significant figures)
 - $[L, U]$ - the exponent range
- Number, e.g. 1.23456×10^{-13} represented as

$$x = \pm \left(d_0 + \frac{d_1}{\beta^1} + \frac{d_2}{\beta^2} + \frac{d_3}{\beta^3} + \dots + \frac{d_{t-1}}{\beta^{t-1}} \right) \beta^e$$

$$0 \leq d_i \leq \beta - 1$$

$$i = 0, 1, 2, \dots, t-1$$

$$L \leq e \leq U$$

- $d_0 d_1 d_2 \dots d_{t-1}$ gives **mantissa** $d_1 d_2 \dots d_{t-1}$, ie behind decimal place
- e is the exponent

Typical Floating Point Systems

- Most computers use binary ($\beta = 2$) arithmetic

System	β	t	L	U
IEEE SP (single precision)	2	24	-126	127
IEEE DP (double precision)	2	53	-1022	1023
Cray XMP	2	48	-16383	16384
HP Calculator	10	12	-499	499
IBM Mainframe	16	6	-64	64

- Note trade-off between t and $[L, U]$
- IEEE by far most common

Normalization

- Floating point system is normalized if leading digit d_0 is always nonzero
 - mantissa m of nonzero floating point number always $1 \leq m \leq \beta$
- Why
 - unique representation of each number
 - no digits wasted on leading zero
 - in binary system the leading bit need not be stored

Properties of Floating Point Systems

- FP number system is finite and discrete
- Number of normalized FP numbers

$$2(\beta - 1)\beta^{t-1}(U - L + 1) + 1$$

- Smallest and largest positive normalized number
 - underflow level $\equiv \text{UFL} = \beta^L$
 - overflow level $\equiv \text{OFL} = \beta^{U+1}(1 - \beta^{-t})$
- FP numbers are equally spaced only between successive powers of β
- Not all real numbers are exactly representable
 - those that are, are called machine numbers

IEEE Single Precision (32 bits)

1	2	3	4	...	9	10	11	12	...	32
Sign	Exponent					Mantissa				

- $\beta = 2$, $t = 24$ (normalized), $L = -126$, $U = 127$
- $2 \times 1 \times 2^{23} \times (127 + 126 + 1) + 1$ numbers
- Exponent bias -127 with valid values of $1 \dots 254$, i.e. $-126 \dots 127$
- Exponent values of 0 and 255 reserved (subnormal and infinity/NAN)

Special Cases : Single Precision

Valid IEEE Single Precision Number

0	0000	1110	1010	0011	1111	1011	0100	111
---	------	------	------	------	------	------	------	-----

IEEE +0

0	0000	0000	0000	0000	0000	0000	0000	000
---	------	------	------	------	------	------	------	-----

IEEE -0

1	0000	0000	0000	0000	0000	0000	0000	000
---	------	------	------	------	------	------	------	-----

IEEE $+\infty$

0	1111	1111	0000	0000	0000	0000	0000	000
---	------	------	------	------	------	------	------	-----

IEEE $-\infty$

1	1111	1111	0000	0000	0000	0000	0000	000
---	------	------	------	------	------	------	------	-----

IEEE Not A Number (NaN)

-	1111	1111	Non Zero					
---	------	------	----------	--	--	--	--	--

Example Floating Point System

- IEEE like 8 bit system
- $\beta = 2$, $t = 5$ (normalized), $L = -2$, $U = 3$, (193 Numbers)
- Normal numbers raw exponent 1 ... 6 with exponent bias of -3
- Example Number

Sign	Exp.	Mantissa
1	100	0110

What is this number in decimal?

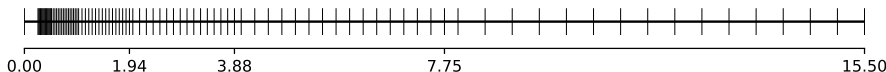
- Sign bit set $\rightarrow$ negative
- Exponent $100_b = 4_{10}$
- Add Exponent bias $4 - 3 = 1$
- Mantissa with leading normalization bit
 $1.0110_b = 1 + (0 \times \frac{1}{2}) + (1 \times \frac{1}{4}) + (1 \times \frac{1}{8}) + (0 \times \frac{1}{16}) = 1.375_{10}$
- Number is $-1.375 \times 2^1 = -2.75$

Example Floating Point System

- $\beta = 2$, $t = 5$ (normalized), $L = -2$, $U = 3$, (193 Numbers)

Sign	d_0	d_1	d_2	d_3	d_4	Base	e	Decimal
0	1	1	1	1	1	1.9375	2^3	15.500
0	1	1	1	1	0	1.8750	2^3	15.000
...								
0	1	1	0	0	1	1.0625	2^{-2}	0.265
0	1	1	0	0	0	1.0000	2^{-2}	0.250
0	0	0	0	0	0	0.0000	0	0.000

- $OFL = (1.1111)_2 \times 2^3 = 15.5_{10}$, $UFL = (1.0000)_2 \times 2^{-2} = 0.25_{10}$
- Note grainy and unequally spaced



Rounding Rules

- Non-machine numbers must be approximated to “nearby” FP number
 - termed rounding and introduces rounding error - denote as $fl(x)$
- Rounding rules
 - chop: truncate x after $(t - 1)$ digit (round toward zero or round down)
 - round to nearest
- IEEE uses rounding to nearest

Machine Precision

- Accuracy of FP system characterized by quantity called “unit round-off error” or “machine precision”, or “machine epsilon”, denoted by ϵ_{mach}
 - chopping: $\epsilon_{\text{mach}} = \beta^{1-t}$
 - nearest: $\epsilon_{\text{mach}} = \beta^{1-t}/2$
- Maximum relative error in representing real number x is given by:

$$\frac{fl(x) - x}{x} \leq \epsilon_{\text{mach}}$$

Machine Precision

- For IEEE FP systems
 - Single Precision: $\epsilon_{\text{mach}} = 2^{-24} \approx 10^{-7}$
 - Double Precision: $\epsilon_{\text{mach}} = 2^{-53} \approx 10^{-16}$
- IEEE SP and DP have ≈ 7 and 16 decimal digits of precision respectively
- Machine precision and UFL are VERY different

$$0 < \text{UFL} < \epsilon_{\text{mach}} < \text{OFL}$$

Subnormal Numbers and Gradual Underflow

- Normalization causes gap around zero



- Subnormal or denormalized numbers permit leading digit to be zero only for minimum exponent.



- Subnormal numbers extend range of magnitude representable, but they have less precision and unit roundoff is the same
- Allows system to exhibit gradual underflow

Errors and Floating Point Arithmetic

- Require, $x \text{ flop } y = fl(x \text{ flop } y)$
 - IEEE systems achieve this as long as $(x \text{ flop } y)$ is within the range of the FP system
- Some familiar laws of real arithmetic are not valid!
- FP addition and multiplication are commutative ($a \times b = b \times a$)
- but NOT associative ($(a + b) + c \neq a + (b + c)$)
- e.g if δ is a small positive number slightly less than ϵ_{mach}
$$(1 + \delta) + \delta = 1 + \delta = 1 \text{ BUT } 1 + (\delta + \delta) > 1$$

Cancellation

- Subtraction of two t -digit numbers with same sign and similar magnitude yields fewer than t digits
 - leading digits of two numbers cancel, e.g.

$$\begin{array}{r} 192.403 \\ -192.275 \\ \hline 0.128 \end{array} = 1.28 \times 10^{-1}$$

- Exact result but cancellation often implies serious loss of information
 - e.g. for small number $\delta < \epsilon_{\text{mach}}$

$$(1 + \delta) - (1 - \delta) = 1 - 1 = 0$$

Correct result of 2δ entirely lost

Error Propagation: Addition/Subtraction

- Addition/subtraction

- add or subtract absolute errors

$$B = 2.0 \pm 0.1 \qquad b = 0.1/2.0 = 0.05$$

$$C = 3.0 \pm 0.3 \qquad c = 0.3/3.0 = 0.10$$

$$A = B + C = 5.0 \pm 0.4 \qquad a = 0.4/5.0 = 0.08$$

- Relative Error Averaged

$$a = \frac{Bb + Cc}{B + C}$$

Error Propagation : Multiplication/Division

- Multiplication/Division

- add or subtract relative errors

$$B = 2.0 \pm 0.1$$

$$b = 0.1/2.0 = 0.05$$

$$C = 3.0 \pm 0.3$$

$$c = 0.3/3.0 = 0.10$$

$$A = B \times C$$

$$A = (2.0 \pm 0.1) \times (3.0 \pm 0.3)$$

$$A = 6.0 \pm (0.6 + 0.3)$$

$$a = 0.9/6.0 = 0.15$$

- Relative errors **summed**

Real Life Error Propagation

- Good algorithms usually propagate errors as $\epsilon\sqrt{N}$
 - N is the number of operations
 - ϵ is the error per operation
- Consider 1GHz computer that can perform 2×10^9 FLOPs
 - Running for 1 hour gives $N = 7.2 \times 10^{12}$ operations
 - $\sqrt{N} \approx 3 \times 10^6$
 - Compare to single precision $\epsilon_{\text{mach}} \approx 10^{-7}$

Virtually all accuracy is lost

- Need to be very careful about error propagation

Polynomial Cancellation Example

```
float poly(float x)
{
    return x*x*x*x*1e3 - x*x*x*2.0 - x*x*4.0e9 + x*2.0e6 + 12.0e9;
}

int main(int argc, char *argv[])
{
    printf("%f\n", poly(2.0e3));
}
```

Output (Should be zero)

-65536000.000000

- Be careful when adding/subtracting large numbers
- Be extra-careful when powers are involved

Polynomial Cancellation Example: Solution 1

- Convert to double to improve precision.
- Gives correct answer yet performance will be degraded.

```
double poly(double x)
{
    return x*x*x*x*1e3 - x*x*x*2.0 - x*x*4.0e9 + x*2.0e6 + 12.0e9;
}

int main(int argc, char *argv[])
{
    printf("%lf\n", poly(2.0e3));
}
```

Output (Should be zero)

0.000000

Polynomial Cancellation Example: Solution 2

- Use change of variables to rescale polynomial domain and range to 0 .. 1
- Gives correct answer yet performance will be degraded.

```
float poly_transformed(float u)
{
    // x = 2000.0*u + 1000.0
    // y = 1.0e15 * v
    return 16.0*u*u*u*u + 32.0*u*u*u + 8.0*u*u - 8.0*u - 3.0;
}

int main(int argc, char *argv[])
{
    float x = 2.0e3;
    float u = (x - 1.0e3)/2.0e3;
    float v = poly_transformed(u);
    float y = 1.0e15 * v;
    printf("%1f\n", y);
}
```

Output (Should be zero)

0.000000

Summary

- Approximation Error
 - Due to the method of approximating derivatives or integrals
 - Due to choice of time steps etc
- Rounding Error
 - Errors due to limited precision of floating point
- Catastrophic Cancellation
 - Loss of precision when adding and/or subtracting numbers with large magnitude differences
- Reducing approximation or rounding errors generally has a cost
 - Change of method (higher order approximations)
 - Change from float to double or long double
- Floating point operations are not associative
 - $a + (b + c) \neq (a + b) + c$
 - When we optimize we may change the order of operations and therefore change the result